

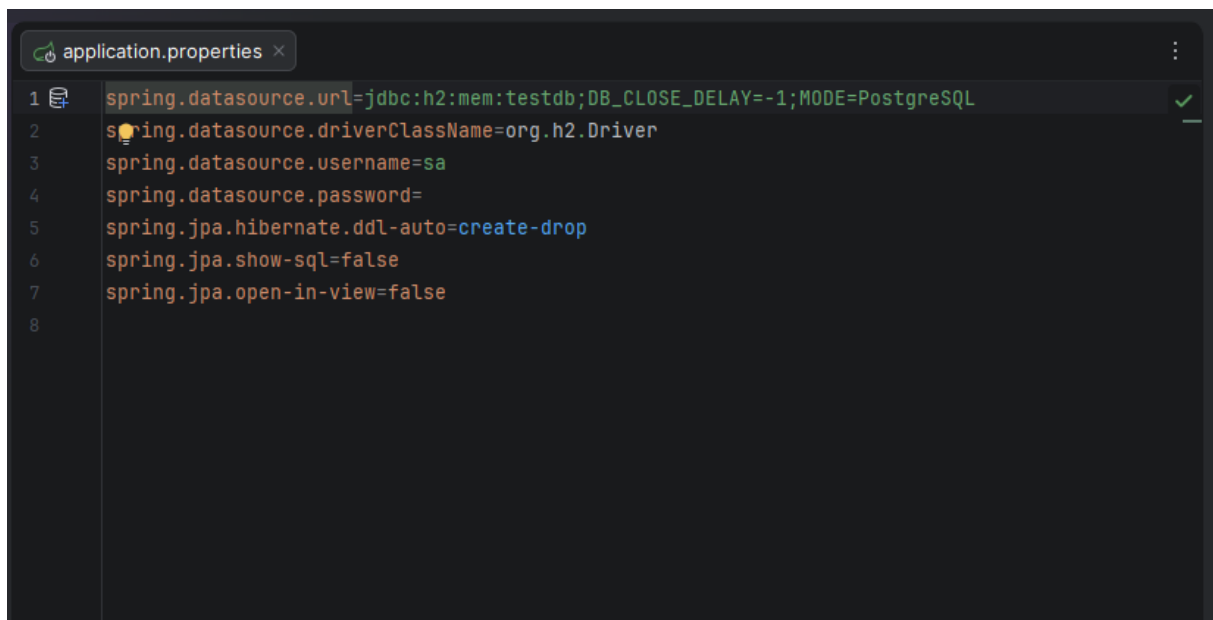
- **Nom de l'étudiant** : KHAMRICH Mohammed
- **Encadre par** : Pr : B.ELBAGHAZAOU

TP1: Application Monolithique avec Spring Boot

Objectifs

- Créer une application Spring Boot monolithique simple
- Développer une API REST CRUD complète
- Comprendre l'architecture en couches (Entity, Repository, Service, Controller)
- Découvrir RestRepository pour simplifier le code
- Identifier les avantages et limites d'un monolithe

Étape 1 : Configuration du Projet



```
application.properties
1 spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;MODE=PostgreSQL
2 spring.datasource.driverClassName=org.h2.Driver
3 spring.datasource.username=sa
4 spring.datasource.password=
5 spring.jpa.hibernate.ddl-auto=create-drop
6 spring.jpa.show-sql=false
7 spring.jpa.open-in-view=false
8
```

Étape 2 : Modèle de Données (L'Entité)

```
package com.ecommerce.monolith.product.model;
```

```

import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.*;
import java.math.BigDecimal;

@Entity
@Table(name = "products")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Product name is required")
    @Column(nullable = false)
    private String name;

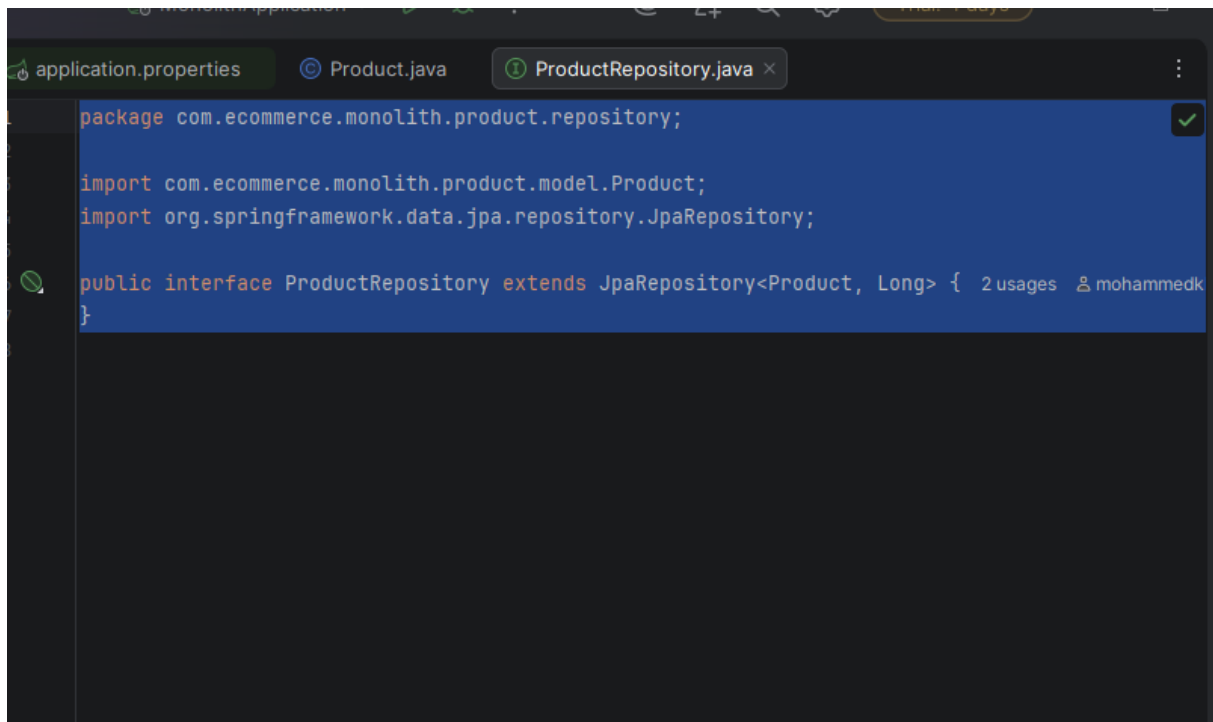
    @Column(length = 1000)
    private String description;

    @NotNull(message = "Price is required")
    @Positive(message = "Price must be greater than 0")
    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal price;

    @NotNull(message = "Stock is required")
    @PositiveOrZero(message = "Stock cannot be negative")
    @Column(nullable = false)
    private Integer stock;
}

```

Étape 3 : Accès aux Données (Repository)

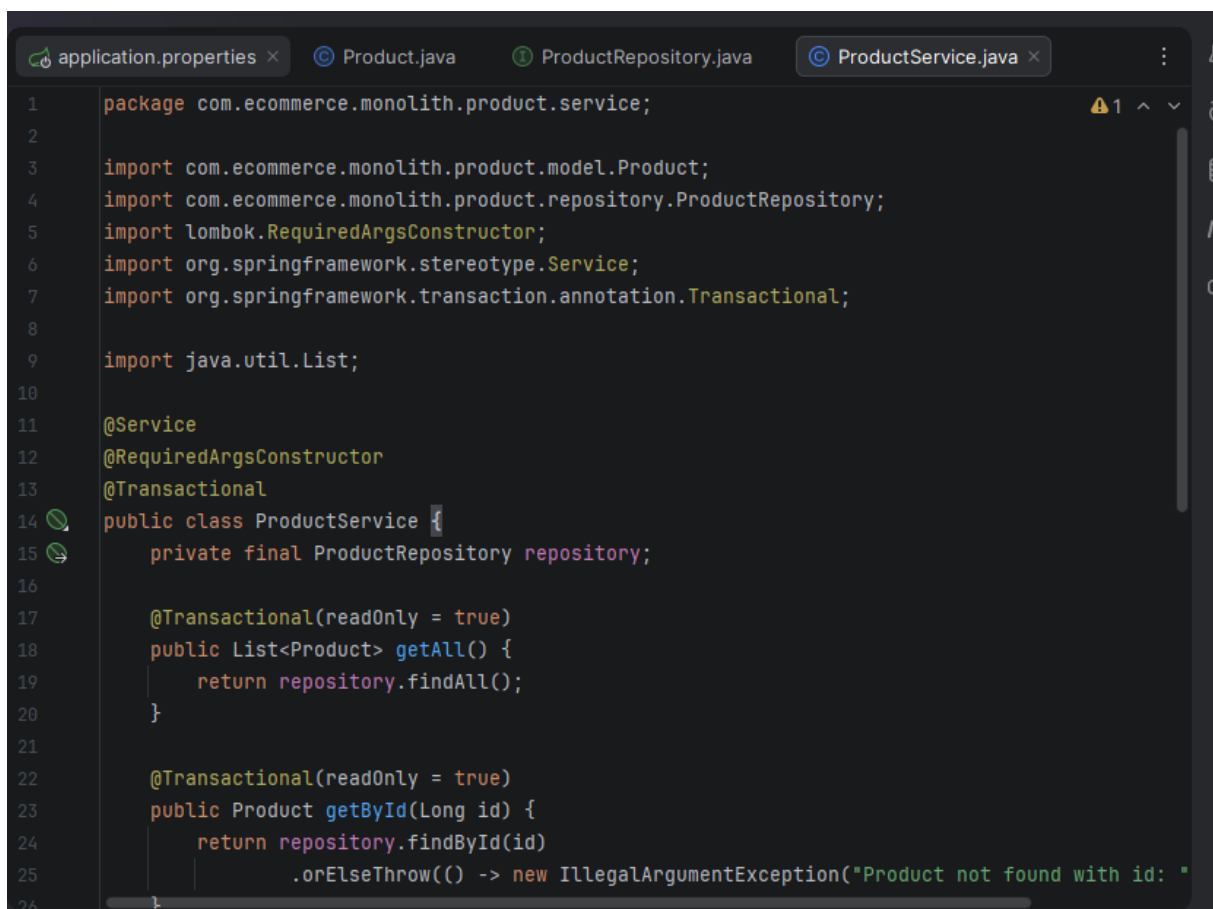


```
package com.ecommerce.monolith.product.repository;

import com.ecommerce.monolith.product.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProductRepository extends JpaRepository<Product, Long> { 2 usages  mohammedk
}
```

Étape 4 : Logique Métier (Service)



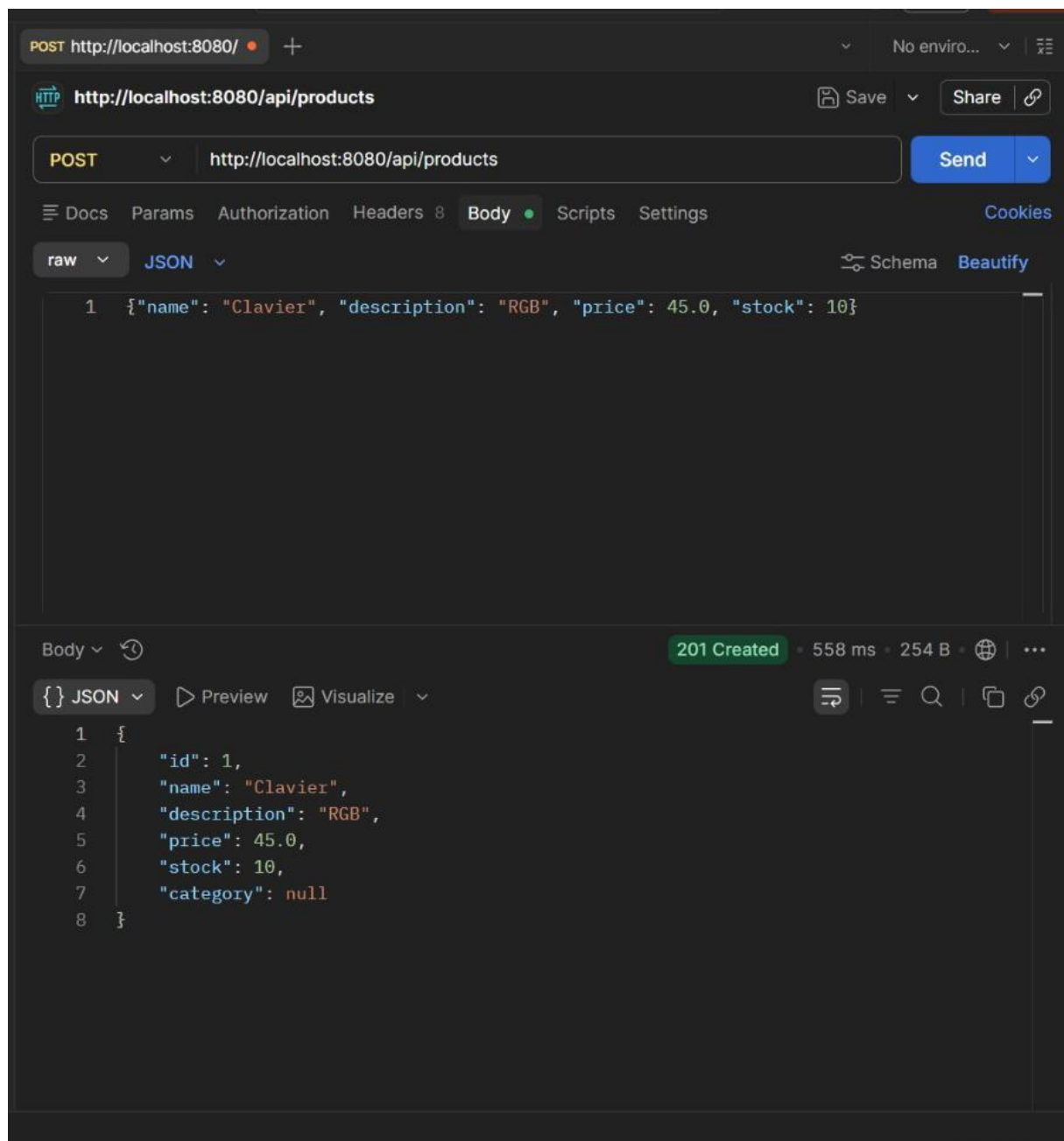
```
1 package com.ecommerce.monolith.product.service;
2
3 import com.ecommerce.monolith.product.model.Product;
4 import com.ecommerce.monolith.product.repository.ProductRepository;
5 import lombok.RequiredArgsConstructor;
6 import org.springframework.stereotype.Service;
7 import org.springframework.transaction.annotation.Transactional;
8
9 import java.util.List;
10
11 @Service
12 @RequiredArgsConstructor
13 @Transactional
14 public class ProductService {
15     private final ProductRepository repository;
16
17     @Transactional(readOnly = true)
18     public List<Product> getAll() {
19         return repository.findAll();
20     }
21
22     @Transactional(readOnly = true)
23     public Product getById(Long id) {
24         return repository.findById(id)
25             .orElseThrow(() -> new IllegalArgumentException("Product not found with id: "
26
```

Étape 5 : Exposition de l'API (Controller)

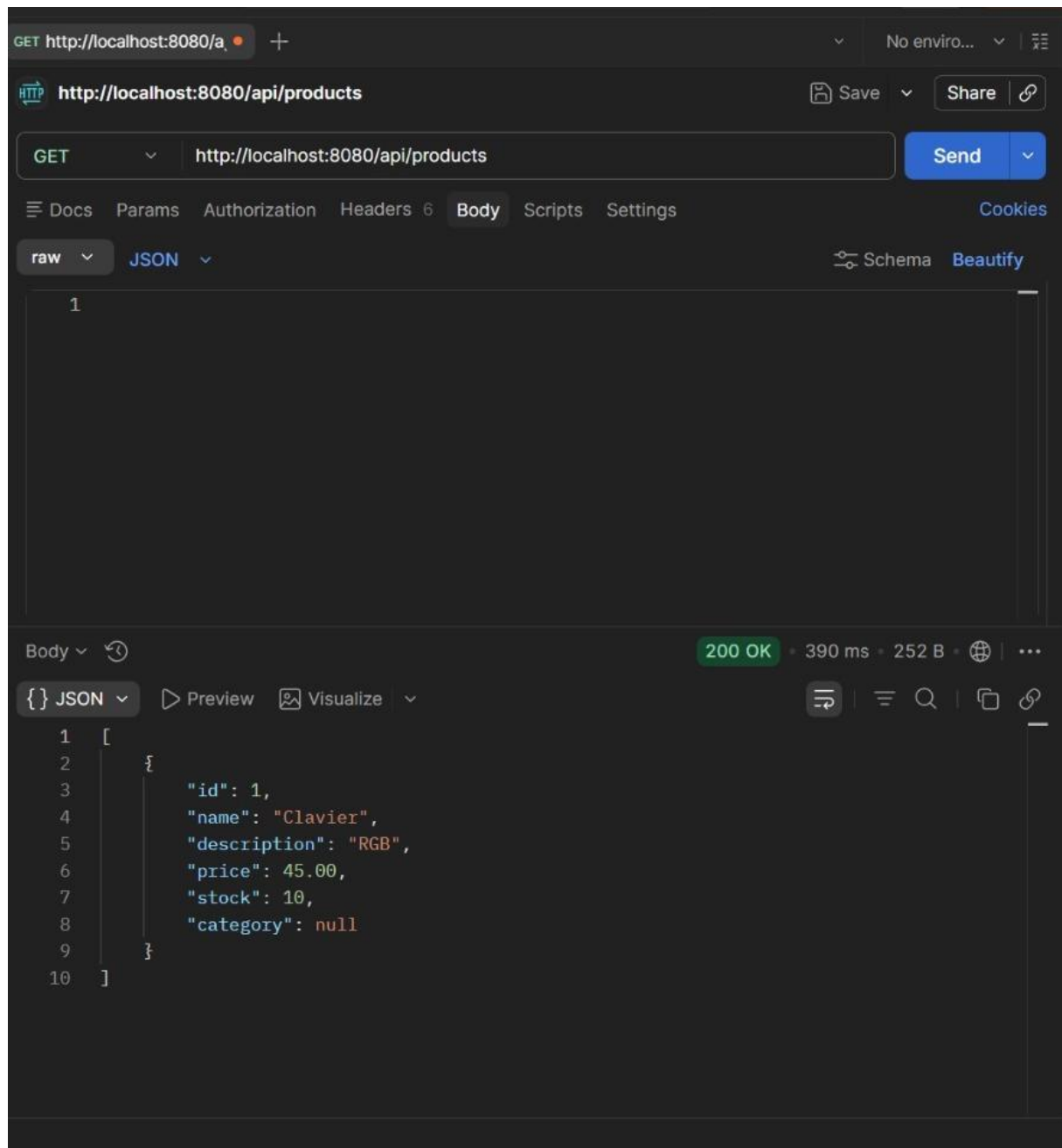
```
ProductController.java x
1 package com.ecommerce.monolith.product.controller;
2
3 import com.ecommerce.monolith.product.model.Product;
4 import com.ecommerce.monolith.product.service.ProductService;
5 import jakarta.validation.Valid;
6 import lombok.RequiredArgsConstructor;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.validation.annotation.Validated;
10 import org.springframework.web.bind.annotation.DeleteMapping;
11 import org.springframework.web.bind.annotation.GetMapping;
12 import org.springframework.web.bind.annotation.PathVariable;
13 import org.springframework.web.bind.annotation.PostMapping;
14 import org.springframework.web.bind.annotation.RequestBody;
15 import org.springframework.web.bind.annotation.RequestMapping;
16 import org.springframework.web.bind.annotation.RestController;
17
18 import java.util.List;
19
20 @RestController @ mohammedkhamrich
21 @RequestMapping("/api/products")
22 @RequiredArgsConstructor
23 @Validated
24 public class ProductController {
25
```

Étape 6 : Tests et Validation (Postman)

POST :



GET :



PUT :

PUT http://localhost:8080/a + No enviro...

HTTP http://localhost:8080/api/products/1 Save Share

PUT http://localhost:8080/api/products/1 Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {"name": "Clavier", "description": "RGB", "price": 60.0, "stock": 10}
```

Body 200 OK • 111 ms • 249 B • ...

{ } JSON Preview Visualize

```
1 {
2   "id": 1,
3   "name": "Clavier",
4   "description": "RGB",
5   "price": 60.0,
6   "stock": 10,
7   "category": null
8 }
```

DELETE :

DEL http://localhost:8080/a

No enviro...

HTTP

http://localhost:8080/api/products/1

Save

Share

DELETE

http://localhost:8080/api/products/1

Send

Docs

Params

Authorization

Headers 8

Body

Scripts

Settings

Cookies

raw

JSON

Schema

Beautify

1

```
{ "name": "Clavier", "description": "RGB", "price": 60.0, "stock": 10 }
```

Body

204 No Content

66 ms

112 B

Raw

Preview

Visualize

1